Object Oriented Programming (CSEG1044)
Unit 05 – Lecture 03 Notes
Wrapped Collections + ‘Collections‘ Utilities + Wrappers

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Lecture Overview **2**

3 Prerequisite Bridge **2**

4 Learning Outcomes **3**

5 Suggested Reading Order **3**

6 Key Terms (Quick) **3**

7 Detailed Notes **3**

7.1 `Collections` utilities (common ones) . . . 3

7.2 Wrapped collections (why/when) . . . 4

7.3 Wrapper classes + autoboxing pitfalls . . . 4

7.4 Class loading basics (intro) . . . 4

8 Running Project Connection **5**

9 Common Mistakes and Confusions **5**

10 Worked Example: utilities + wrappers + autoboxing **5**

11 Demo / Observation Task **6**

12 Practice Check (with brief solutions) **6**

13 Self-Study Practice (no solutions) **6**

14 Lab Alignment (recommended) **6**

15 Further Study **6**

1 Lecture Snapshot

- **Unit focus:** Collections Framework and Wrapper Classes
- **Main new ideas:** `Collections` utilities (common ones), Wrapped collections (why/when), Wrapper classes + autoboxing pitfalls
- **Course thread:** Use Wrapped Collections + ‘Collections‘ Utilities + Wrappers to manage in-memory project data more cleanly and predictably inside Campus Resource Manager.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

2 Lecture Overview

This lecture collects three practical topics that show up in day-to-day Java code:

- utility methods in `java.util.Collections`,
- wrapper “views” around collections (unmodifiable, synchronized, checked),
- wrapper classes (`Integer`, `Double`, ...) and autoboxing pitfalls.

We also briefly introduce the class loading idea from the syllabus.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

3 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **Collection interfaces**: Utilities and wrappers only make sense once List, Set, and Map roles are already clear. Main reference: `Unit 05 / Lecture 01 slides/notes`.
- **Primitive and object types**: Wrapper classes connect directly to the type-conversion ideas from Unit 01. Main reference: `Unit 01 / Lecture 03 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

4 Learning Outcomes

After this lecture, you should be able to:

- Use common `Collections` utilities (sort, min/max, shuffle, reverse).
- Use wrapped collections (unmodifiable/synchronized/checked) appropriately.
- Avoid common wrapper/autoboxing pitfalls (and know class loading basics).

5 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.
2. Study the central explanation in this order: *`Collections` utilities (common ones) → Wrapped collections (why/when) → Wrapper classes + autoboxing pitfalls → Class loading basics (intro).*
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

6 Key Terms (Quick)

- **Utility class**: class with static helper methods (e.g., `Collections`).
- **Unmodifiable view**: wrapper that blocks structural modification.
- **Synchronized wrapper**: wrapper that adds basic synchronization around operations.
- **Checked wrapper**: wrapper that enforces runtime type checks on adds.
- **Wrapper class**: object form of a primitive (e.g., `Integer` for `int`).
- **Autoboxing/unboxing**: automatic conversion between primitive and wrapper.
- **Class loading**: JVM loads class bytecode into memory when needed.

7 Detailed Notes

7.1 `Collections` utilities (common ones)

`java.util.Collections` provides algorithms for collections:

- `sort(list)` or `sort(list, comparator)`
- `reverse(list)`

- `shuffle(list)`
- `min(list)`, `max(list)`
- `frequency(collection, element)`
- `binarySearch(list, key)` (requires sorted list)

7.2 Wrapped collections (why/when)

Wrapped collections are "views" (wrappers) around an existing collection. They are used to enforce constraints without copying:

- `unmodifiableList`: when you want to expose read-only data to callers.
- `synchronizedList`: basic thread-safety wrapper (older style; modern code may use concurrent collections).
- `checkedList`: runtime type checking (useful when mixing legacy/raw types).

Important: "unmodifiable" means you cannot change structure via the wrapper, but if the underlying list changes, the view reflects it.

7.3 Wrapper classes + autoboxing pitfalls

Wrapper classes are needed because generics work with reference types, not primitives:

- `List<Integer>` not `List<int>`

Pitfalls:

- Unboxing null → `NullPointerException`

  Example: `Integer x = null; int y = x;` crashes.
- Using `==` for wrapper comparisons can be confusing.

  Prefer `equals()` or compare primitives.
- Performance: excessive boxing/unboxing in tight loops can slow code.

7.4 Class loading basics (intro)

The JVM loads classes when they are needed (first time referenced). `Class.forName("com.mysql.cj.jdbc.Driver")` is sometimes used to load a JDBC driver explicitly (older pattern). If a class is missing, you may see `ClassNotFoundException`.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

8 Running Project Connection

Use Wrapped Collections + ‘Collections‘ Utilities + Wrappers to manage in-memory project data more cleanly and predictably inside Campus Resource Manager.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today’s idea should live: model, service, DAO, UI, or team workflow.
- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

9 Common Mistakes and Confusions

- Assuming wrapper objects behave exactly like primitives in every context.
- Treating synchronized wrappers as a full concurrency design.
- Forgetting that an unmodifiable view can still reflect external changes to the backing collection.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

10 Worked Example: utilities + wrappers + autoboxing

```java
import java.util.*;

public class Demo {
    public static void main(String[] args) {
        List<Integer> a = new ArrayList<>(Arrays.asList(3, 1, 2));
        Collections.sort(a);
        System.out.println("Sorted: " + a);
        System.out.println("Min=" + Collections.min(a) + ", Max=" + Collections.max(a));

        List<Integer> ro = Collections.unmodifiableList(a);
        // ro.add(99); // UnsupportedOperationException

        List checked = Collections.checkedList(new ArrayList(), String.class);
        checked.add("ok");
        // checked.add(123); // ClassCastException at add-time

        Integer x = null;
        // int y = x; // NullPointerException (unboxing null)
    }
}
```

11 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *`Collections` utilities (common ones)* once without looking at the solution first.
- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

12 Practice Check (with brief solutions)

1. What exception occurs if you try to add to an unmodifiable list?

   Answer: `UnsupportedOperationException`.

2. Why do we need wrapper classes with collections?

   Answer: generics require reference types; primitives cannot be used directly.

3. What happens if you unbox `Integer x = null`?

   Answer: `NullPointerException`.

13 Self-Study Practice (no solutions)

1. Shuffle a list of integers and print the result.
2. Create an unmodifiable view of a list and test which operations fail.
3. Write a program that reads integers into a `List<Integer>` and then finds min/max using `Collections`.

14 Lab Alignment (recommended)

- Exp-18: Collections Framework
- Exp-19: Wrapper Classes

15 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.

- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: The Complete Reference* (12e) (Collections Framework + wrapper classes).; Oracle Java docs: Collections Framework overview (accessed April 1, 2026).; Java SE API docs (`java.util` collections; wrapper classes in `java.lang`) (accessed April 1, 2026).

16 Exit Questions (with sample answers)

Q1. What is the difference between `Collections` and `Collection`?

Sample answer: `Collection` is an interface representing a group of elements. `Collections` is a utility class containing static helper methods for working with collections.

Q2. Why can unboxing a null `Integer` crash?

Sample answer: If `x` is null, converting it to a primitive triggers `NullPointerException`.

Q3. What risk appears when autoboxing meets null values?

Sample answer: Java may try to unbox null into a primitive, which throws a NullPointerException at runtime.